

Midterm Project Report: Asynchronous Deterministic Coupling for Distributed Spatial Simulations

Hritvik

Introduction

Motivation of the project: Large-scale Parallel Discrete Event Simulations (PDES), particularly high-resolution discrete spatial grids, face a fundamental scalability limit: the synchronization bottleneck at partition boundaries. When simulating emergent physical interactions occurring across a spatial boundary, nodes must coordinate state transitions to ensure physical consistency. Standard distributed architectures rely on conservative synchronization, where nodes block execution until all incoming messages from neighbors are verified. In a high-frequency environment, this assumes a “zero-latency” network, leading to significant compute-resource idling. Alternatively, optimistic approaches (such as the Time Warp system) avoid blocking by allowing speculative execution, but incur massive memory overhead by saving historical state snapshots to facilitate rollbacks when boundaries diverge. This project is motivated by the need to support highly dense spatial models on distributed nodes that are limited by local memory capacity, such as autonomous robot fleets or drone swarms attempting to compute shared spatial trajectories in real-time.

Project objectives: The primary objective of this project is to implement and evaluate a “Memoryless Optimistic Protocol.” Assuming a reliable network, this project introduces an Asynchronous Rule-Based Coupling mechanism. It aims to demonstrate that distributed worker nodes can independently compute state transitions and resolve spatial boundary conflicts locally without memory rollbacks or secondary network negotiations.

Main contributions: The core contribution of this work is the design and implementation of a static, globally shared conflict resolution hierarchy that guarantees state convergence across distributed partitions. Instead of negotiating with peers or rolling back to historical states, nodes apply a deterministic priority algorithm to divergent boundary cells. This transforms distributed network conflicts into local deterministic resolutions in $\mathcal{O}(1)$ time.

Deliverables:

- **Source Code:** A Python-based master-worker architecture utilizing gRPC, featuring a 2D spatial simulation engine acting as the testbed (a Forest Fire Cellular Automata).
- **Technical Demonstration:** A system log demonstration showing independent state convergence across distributed partitions during asynchronous simulation ticks.
- **Throughput Analysis & Evaluation:** Empirical benchmarks comparing the computational tick-rate (throughput) of this optimistic deterministic coupling against traditional conservative blocking models under artificially injected network latency.
- **Final Report:** Detailed analysis of the methodology, algorithm design, evaluation metrics, and performance gains.

Related Work

The challenge of boundary synchronization in PDES has been extensively studied. Foundational PDES models by Fujimoto [4] and the Time Warp System by Carothers et al. [5] provide the baseline metrics for evaluating the trade-offs between conservative blocking (which wastes compute cycles) and optimistic execution (which wastes memory). Recent work by Andelfinger and Uhrmacher [1] proposes synchronous speculative simulation for tightly coupled agents, highlighting the ongoing challenges of maintaining consistency across partitions without severe overhead. Similarly, distributed simulation frameworks for digital twins continue to rely heavily on optimistic PDES engines but face severe memory and state consistency bottlenecks [2]. In spatial simulations specifically, recent research on parallel cellular automata emphasizes boundary coordination and load balancing challenges [3]. These works collectively underscore the necessity for a lightweight, memory-efficient optimistic synchronization strategy.

Progresses

Significant progress has been made on the core architecture and the underlying physics engine. The project has advanced from the theoretical design phase to a functional Proof of Concept (PoC) running on a local cluster.

1. Simulation Engine Implementation: Developed a strictly deterministic Forest Fire Cellular Automata to act as the simulation testbed. The ruleset relies on three states (`EMPTY=0`, `TREE=1`, `FIRE=2`) and guarantees predictable simulation outcomes based on neighbor states. The transition rules (Ignition, Burnout, and Deterministic Growth) ensure that the simulation’s physical state is a pure function of its previous tick, isolating any state divergence strictly to network asynchrony.

2. Deterministic Conflict Engine: Constructed the core `resolve_boundary_conflict` local function. If network latency causes nodes to disagree on a boundary (e.g., Node A computes a boundary cell as `TREE` while Node B’s delayed packet reports `FIRE`), both nodes apply a strict transitive hierarchy (`FIRE > TREE > EMPTY`) upon receiving the peer’s data. Both nodes independently apply `max(local_state, peer_state)`, instantly converging on the highest-energy state without secondary network communication.

3. Distributed gRPC Network Architecture: Successfully scaffolded and deployed a Python-based distributed cluster. Auto-generated protobuf schemas (`TickRequest`, `BoundaryData`, `BoundaryAck`) route highly compressed binary data between a centralized Master orchestration node and peer-to-peer Worker nodes over TCP. The Master strictly acts as a global clock tracking computational throughput, while the Workers handle the heavy lifting of the cellular automata grid and peer-to-peer hash exchanges.

4. Empirical Validation and Conflict Testing: Conducted local cluster testing across separate process ports. Artificial boundary conflicts were manually injected into the worker nodes to simulate severe network lag. System logs successfully demonstrate the nodes independently detecting the conflict, applying the $\mathcal{O}(1)$ deterministic resolution, and achieving perfect physical state convergence within two computational ticks without rolling back memory or dropping simulation steps.

AI Usage

AI was used in the following ways:

- Assisting with writing code schemas and running commands to convert into python code
- Identifying pitfalls and potential infinite loops in state transitions
- Helping write latex code and wording/formatting of documents to be more in line with expectations of a research document

Lessons Learned So Far

Refining the Mechanics: Initially, the proposal utilized an abstract “Solid > Liquid > Gas” hierarchy. I realized this analogy obfuscated the actual computer science mechanics and risked introducing cyclical (Rock-Paper-Scissors) infinite loops if not strictly enforced. Pivoting to a deterministic Cellular Automata model (Forest Fire) provided a much clearer, mathematically sound, and strictly transitive vehicle to test the network protocol.

Network Scope and Reliability: Following initial feedback from the instructor, the scope of the communication layer was heavily refined. Instead of testing UDP and building complex packet-recovery or retry algorithms, the architecture now strictly utilizes gRPC. By assuming a reliable network (no dropped packets), I can focus entirely on the core research question: resolving latency-induced asynchrony using deterministic rules rather than debugging lower-level network transport issues.

Action Plan and TimeLine

- **Week 8:** Refine the Master node’s orchestration logic to simulate artificial network latency dynamically. This will force the nodes to drift out of sync organically rather than relying on manually injected grid conflicts.
Develop the baseline “Conservative Blocking” network model. This will serve as the strict control group for our evaluation, requiring workers to idle until boundary data is received.
- **Week 9:** Conduct rigorous stress testing. Benchmark the computational tick-rate (throughput) of the Deterministic Optimistic model against the Conservative model under varying latency configurations (e.g., 10ms, 50ms, 100ms delays).
- **Week 9 & 10:** Aggregate throughput data, generate performance graphs, record the final technical demonstration video, and compile the final evaluation report.

References

- [1] P. Andelfinger and A. Uhrmacher, “Synchronous speculative simulation of tightly coupled agents in continuous time on CPUs and GPUs,” *SIMULATION*, 2023.
- [2] Z. Huang et al., “Distributed Simulation for Digital Twins of Large-Scale Real-World DiffServ-Based Networks,” *arXiv preprint arXiv:2405.20815*, 2024.
- [3] W. Xia et al., “Dynamic Load Balancing Based on Hypergraph Partitioning for Parallel Geospatial Cellular Automata Models,” *ISPRS International Journal of Geo-Information*, 2025.
- [4] R. M. Fujimoto, “Parallel Discrete Event Simulation,” *ACM Computing Surveys*, vol. 22, no. 1, pp. 1-45, 1990.
- [5] C. D. Carothers, D. Bauer, and S. Pearce, “ROSS: A High-Performance, Low-Memory, Modular Time Warp System,” *Journal of Parallel and Distributed Computing*, 2002.